

Predicting Cache Evolution: The Growing Capacity and Latency Cost of CPU Caching

Nick Ngo Kevin Chen

ECE 592 – Fall 2026

Motivation and Experimental Question

Question: Can a cache hierarchy's geometry, latency, and inclusion behavior be reverse engineered from timing alone, and does that hierarchy change predictably across CPU generations and vendors?

Three-phase discipline (non-negotiable)

- 1 **Phase I – timing only:** no PMU, no cache-topology files, no vendor tables.
- 2 **Phase II – counter & literature verification:** perf, lscpu, Agner Fog / vendor docs.
- 3 **Phase III – Hazel:** a lab-only prediction is frozen *before* touching the held-out HPC cluster.

Six timing-only targets: cache-level/capacity, line size, associativity, hit latency, miss/next-level latency, inclusion/exclusion – one shared, portable benchmark binary (`cache_bench`) implements all six.

Experimental Platforms: 8 Lab Machines + Hazel

Machine	Vendor/ISA	Microarch.	Year	Node
Sunbird	Intel x86-64	Haswell	2014	22 nm
Charnwood	Intel x86-64	Skylake	2015	14 nm
Ookay	Intel x86-64	Kaby Lake	2017	14 nm
Upgrade	Intel x86-64	Coffee Lake	2017	14 nm
Crux	Intel x86-64	Coffee Lake Refresh	2019	14 nm
Skylark	AMD x86-64	Zen 2	2019	7 nm
Thunderbird	Ampere AArch64	Neoverse N1	2020	7 nm
Artemisia	Intel x86-64	Sapphire Rapids	2023	10 nm
Hazel-Haswell	Intel x86-64 (HPC)	Haswell-EP	2014	22 nm

Coverage: 2014–2023, 3 vendors (Intel/AMD/Ampere), 2 ISAs (x86-64/AArch64), desktop + server class, plus 1 held-out Slurm HPC generation.

Shared Measurement Infrastructure

Dependent pointer-chase primitive

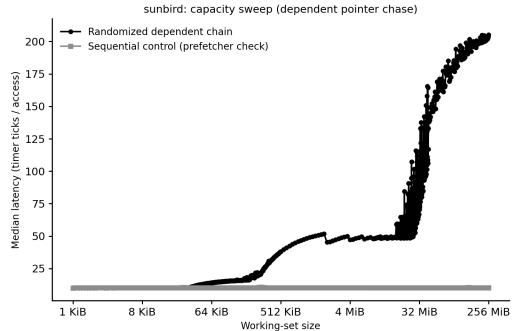
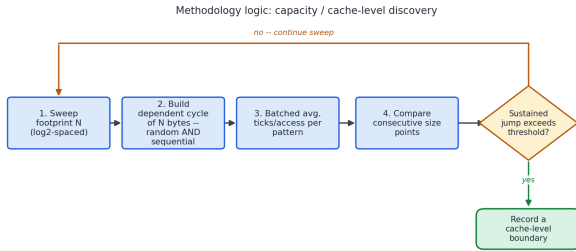
- Singly-linked cycle of fixed-size nodes; load $i+1$'s address is the *value* of load i
- Forces true load-to-use latency – no reorder/prefetch across the chain
- Serialized cycle counters: LFENCE+RDTSC(P) (x86), DSB+ISB+CNTVCT_ELO (ARM)

One binary, `cache_bench`, dispatches all six experiments via `--experiment`; identical code runs unmodified on x86-64 and AArch64.

Batching & controls

- Batches of 1000 accesses amortize timer overhead; $\geq 1,000,000$ timed accesses/point
- Every sweep run at *random* (chase) **and** *sequential* (prefetcher control) patterns
- Independent-load timer exposes MLP as a sanity check on the dependent chain

Result: Capacity / Cache-Level Discovery



Finding

A reproducible, sustained step in random-pattern ticks/access (absent on the sequential control) marks a level boundary; L1D/L2 matched PMU exactly on all 8 machines, LLC on 5/8.

Result: Cache Line Size

Methodology logic: cache line-size inference (two independent methods)

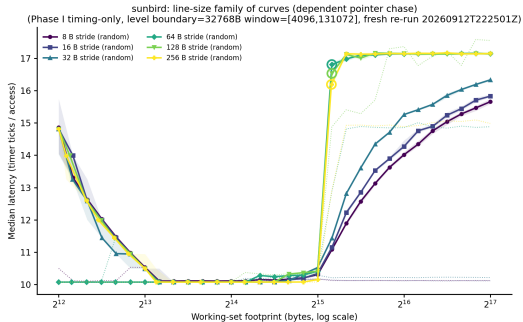
Method A -- single-curve



Method B -- family-of-curves



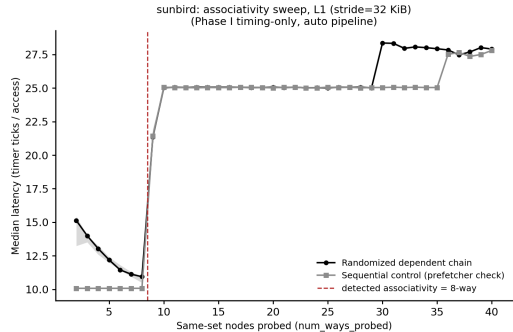
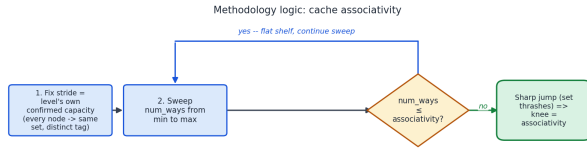
agree?



Finding

Two independently designed methods (stride-saturation and footprint-family) agree at **64 B on 8 of 8 machines** – the single most consistent result in the project (one exception: Skylark's LLC region measures 128 B by both methods).

Result: Cache Associativity – and an Honest Negative Result

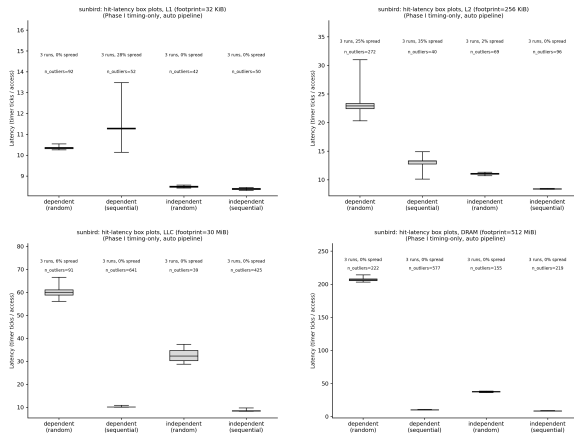
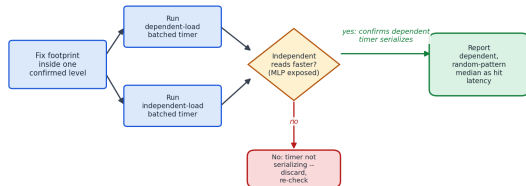


Finding

L1 associativity is confirmed cleanly everywhere (8-way x86, 4-way ARM). **L2/LLC are blocked by a real confound:** byte-capacity candidates spanning up to $32\times$ break at the *same* small num_ways ($\approx 9-10$) regardless of capacity – a shared, capacity-independent structure (leading hypothesis: DTLB aliasing), not real cache signal.

Result: Cache Hit Latency

Methodology logic: cache hit-latency inference



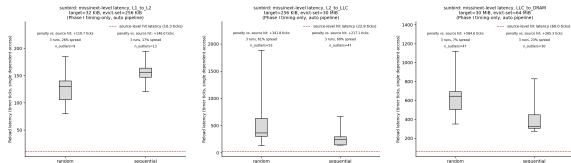
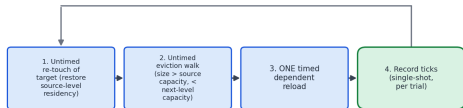
Finding

Dependent/random median at a confirmed single-level footprint, validated by an independent-load control reading faster at every level – Sunbird: a clean monotonic 10.4 / 26.8 / 58.4 / 207 tick ladder

Result: Miss / Next-Level Latency

Methodology logic: miss / next-level-latency inference

repeat per trial

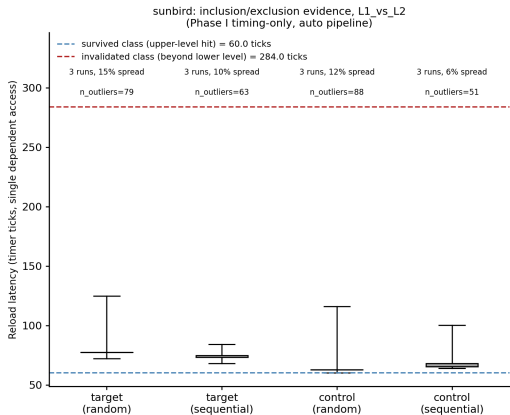
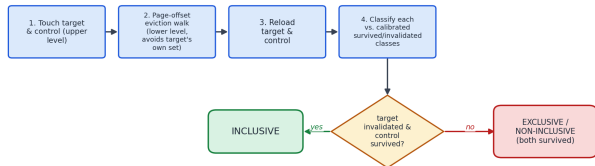


Finding

Single-shot reload after an untimed eviction walk gives an increasing 124/284/622-tick sequence (Sunbird) – but single-shot timing carries a fixed ≈ 64 –85 tick overhead (isolated via a control eviction set), so this is reload latency *plus* overhead, not a clean incremental number.

Result: Inclusion and Exclusion Behavior

Methodology logic: inclusion / exclusion behavior



Finding

A page-offset eviction walk structurally avoids the target's own upper-level set; classifying reload latency against survived/invalidated calibration gives **L1 vs. L2 non-inclusive on 6 of 8 machines** – the one

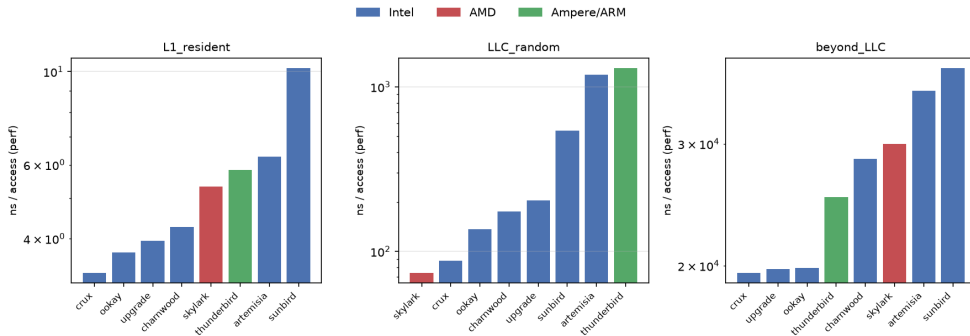
Phase II: Performance Counter Verification

Machine	L1D size	L1D assoc.	L2 assoc.	LLC size	LLC assoc.
Sunbird	✓	✓	✓	✓	✗ (20w)
Charnwood	✓	✓	✗ (4w)	✓	✗ (16w)
Ookay	✓	✓	✗ (4w)	✓	✗ (16w)
Upgrade	✓	✓	✗ (4w)	✓	✗ (16w)
Crux	✓	✓	✗ (4w)	✗ (1.5x)	✗ (12w)
Skylark	✓	✓	✓	✗ (2x)	✗ (16w)
Thunderbird	✓	✓	✗ (8w)	n/a	lit. 16w
Artemisia	✓	✓	✓	✗ (1.75x)	✗ (15w, close)

Headline

L1D matched ground truth on 8 of 8 machines, zero exceptions – the strongest validation in the project. The confound-blocked L2/LLC associativity guess was right on only 3/8 and 1/8 machines respectively, confirming Section 4's own low-confidence flag.

Eight Performance Counters Across Generations

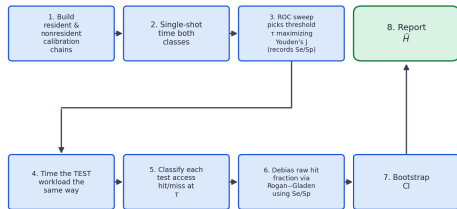


Finding

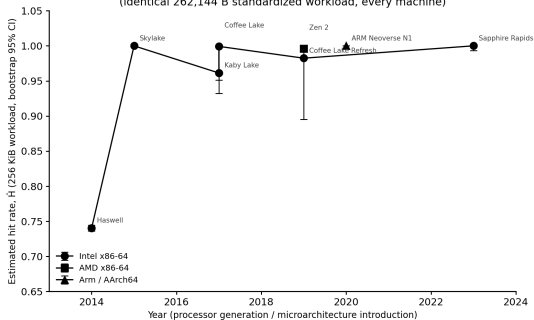
Per-access latency ranked ascending, colored by vendor: Intel machines cluster tightest; Skylark (AMD) shows a striking dTLB-miss anomaly (2–4 orders of magnitude lower than every other machine at LLC/beyond-LLC footprints) – a real architectural TLB-reach difference, reproducible across two benchmark tiers.

Software-Only Cache Hit-Rate Metric

Methodology logic: software-only cache hit-rate metric



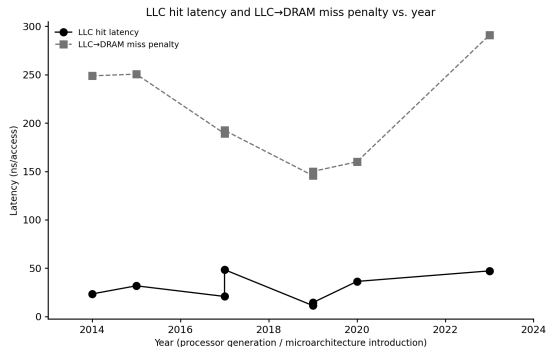
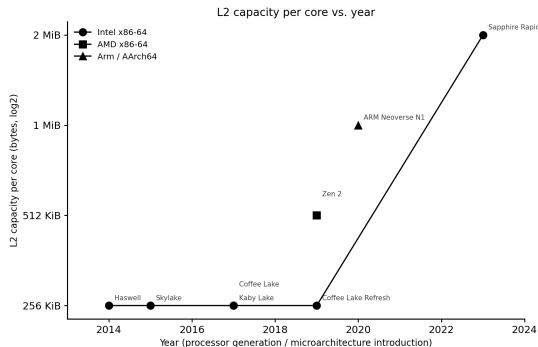
Software-only timing-derived hit rate vs. year
(identical 262,144 B standardized workload, every machine)



Finding

A single self-calibrated latency threshold + Rogan-Gladen debiasing detects L1/L2-scale residency well (10–20% median PMU relative error) but **fails at LLC/DRAM scale** (60–100% error) – a disclosed single-threshold limitation, not a bug found after the fact. At a standardized 256 KiB workload, 7 of 8 machines read ≈ 1.0 ; only Sunbird is a statistically real outlier (a boundary/conflict effect).

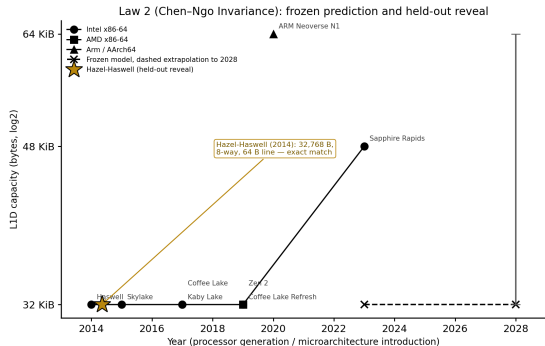
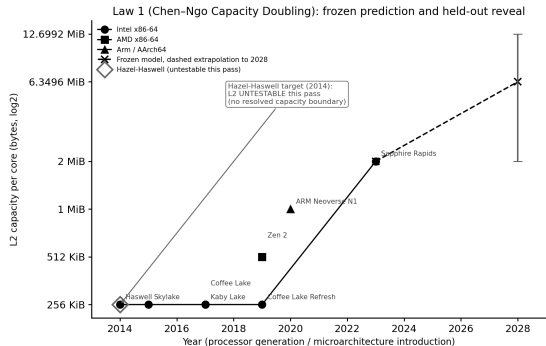
Cache Evolution: Moore-Style Trends



Finding

Per-core L2 capacity grows in a genuine, if *step-wise* (not smooth), doubling pattern: flat at 256 KiB for 5 years, then three back-to-back doublings in 4 years. LLC hit latency does **not** track LLC capacity monotonically – topology (interconnect, core count) dominates once sharing-domain differences are PMU-corrected.

Our Laws: Frozen Predictions vs. Held-Out Hazel

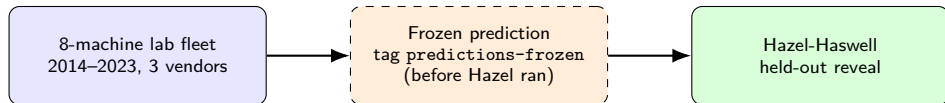


Finding

Law 1 (Doubling): $C(t) = 256 \text{ KiB} \cdot 2^{(t-2014)/3}$, fit Sunbird 2014 $\rightarrow$ Artemisia 2023 ($8\times/9\text{ yr}$, step-wise). Dashed = frozen extrapolation to 2028 ($\sim 6.35 \text{ MiB}$). Hazel's own L2 boundary never resolved – **untestable this pass**.

Law 2 (Invariance): L1D capacity/assoc./line size constant across a decade, 3 vendors (32,768 B / 8-way / 64 B, 6 of 8 machines); dashed line reverts to that majority value, not a fit through Artemisia's outlier. Hazel-Haswell (star) matched exactly – **supported**.

Hazel Held-Out Validation



Quantity	Frozen prediction	Hazel measured	Verdict
L1D capacity/assoc./line size	32,768 B / 8-way / 64 B	exact match, all 3	Supported
LLC capacity	~31.46 MB	~22–24 MB	Weakened (~25–30% low, SKU effect)
L2 capacity/assoc.	256 KiB / 8-way	no clean boundary	Untestable this pass
Inclusion, L1 vs. LLC (skip-level)	lean inclusive	non-inclusive	Falsified

8 of 15 predicted quantities cleanly supported; a genuinely mixed, honestly reported result.

Limitations and Contributions

Limitations

- L2/LLC associativity unresolved by timing alone above L1 (shared confound)
- Miss-latency single-shot overhead not auto-subtracted
- Software hit-rate estimator: L1/L2-scale only, fails at LLC/DRAM
- Hazel: full suite on 1 of 9 available generations (haswell); Phase II not yet run there

Conclusion: L1D geometry is the one part of the memory hierarchy that has not needed to change across a decade and three competing vendors.

Contributions

- Nick Ngo: capacity/associativity/latency/inclusion engines, all lab sweeps, PMU verification (6 machines), chronological analysis & cache laws
- Kevin Chen: line-size engine (both methods), software hit-rate estimator, timer/portability layer, eight-counter study, entire Hazel HPC pipeline